GUIA DE APRESENTAÇÃO | 7 INTEGRANTES

Simulador SIC/XE em C

Falas prontas, arquivos na tela e trechos para destacar

Um roteiro para mostrar **como o código funciona**, sem ler funções inteiras nem atribuir ao projeto funcionalidades que ainda não estão completas.

ORDEM SUGERIDA E DURAÇÃO

01. Malone - Estrutura de memória e registradores	**1min10s**
02. Gabriel Silveira - Decodificação e modos de endereçamento	**1min20s**
03. Gabriel Britto - Instruções lógicas, aritméticas e ponto flutuante	**1min20s**
04. João Victor - Movimentação de dados e desvios	**1min25s**
05. Yago - Carregamento pela interface e montador *(sugestão)*	**1min25s**
06. Vitor - Apresentação gráfica da memória	**1min10s**
07. André - Layout principal e integração do sistema	**1min40s**

Meta de gravação: cerca de 9min30s.
Os tempos são estimativas, incluindo as trocas de tela. Ensaiem uma vez e ajustem as pausas. Cada integrante tem uma página com a fala e os pontos de código.

QUAL VERSÃO ABRIR

Use a pasta **outputs/simulador-sicxe-interface** deste projeto. Todos os caminhos nas próximas páginas são relativos a essa pasta. As linhas foram conferidas na versão de **21/10/2026** e podem mudar se os arquivos forem editados.

Durante a fala: deixe as abas abertas, ative os números de linha e use Ctrl+G para ir ao trecho. Nos blocos longos, selecione apenas a expressão indicada no roteiro. Não é necessário ler o código palavra por palavra.

Atenção à divisão: a função específica do Yago não foi informada. Sua fala de carregamento pela interface e montador é uma sugestão para o vídeo, a confirmar pelo grupo. O roteiro descreve responsabilidades de apresentação, não comprova autoria individual.

PARTE 01 | TEMPO SUGERIDO: 1MIN10S

Malone

Estrutura de memória e registradores

ABRIR NA TELA
sicxe_config.h
memoria.h
memoria.c

TRECHOS PARA MOSTRAR, NESTA ORDEM

1. sicxe_config.h | linhas 5-7
Tamanhos e máscaras. Mostre MEM_TAMANHO = 4096 e MASK24. A memória tem 4 KiB; a máscara limita valores a 24 bits.

2. memoria.h | linhas 7-22
Banco de registradores. Aponte os índices e a estrutura Registrador. uint64_t é o recipiente em C, não a largura de todos os registradores da máquina.

3. memoria.c | linhas 7-16
Estado inicial. inicializar_memoria zera os bytes e o banco de registradores.

4. memoria.c | linhas 18-31
Leitura e escrita de uma palavra. Destaque os deslocamentos de 16 e 8 bits: três bytes formam uma palavra de 24 bits, com o mais significativo primeiro.

FALA SUGERIDA

Eu vou explicar a base de memória e registradores do simulador. No arquivo sicxe_config.h, definimos uma memória de 4096 bytes, ou 4 KiB, para esta implementação. A máscara MASK24 mantém os valores inteiros dentro dos 24 bits usados pela arquitetura.

Em memoria.h, cada registrador tem um índice, como A, X, L e PC. O campo valor usa uint64_t para armazenar os dados em C, mas isso não significa que todos os registradores simulados tenham 64 bits.

Em memoria.c, a inicialização zera a memória e os registradores. A função ler_word_24bits junta três bytes usando deslocamentos de bits. A escrita faz o caminho inverso e separa a palavra em três bytes, colocando o byte mais significativo primeiro.

Essas funções também verificam se os três bytes cabem na memória. Essa estrutura é compartilhada pelas instruções e pela interface, então as duas partes trabalham sobre o mesmo estado. Agora o Gabriel Silveira explica como os bytes são interpretados.

LEMBRETE PARA A GRAVAÇÃO
Não diga que a memória é de 1 MiB: este projeto está configurado com 4096 bytes. As rotinas de palavra retornam zero ou não escrevem quando o endereço é inválido.

PARTE 02 | TEMPO SUGERIDO: 1MIN20S

Gabriel Silveira

Decodificação e modos de endereçamento

ABRIR NA TELA
decodificador.c

TRECHOS PARA MOSTRAR, NESTA ORDEM

1. decodificador.c | linhas 277-300
Separar os campos. No formato 2, destaque r1 e r2. Depois mostre opcode = byte1 & 0xFC e a extração de n, i, x, b, p e e.

2. decodificador.c | linhas 319-355
Tamanho e próximo PC. Mostre os formatos 3 e 4: deslocamento de 12 bits ou endereço de 20 bits, com PC avançando 3 ou 4 bytes.

3. decodificador.c | linhas 149-177
Endereço relativo e indexado. A linha 154 usa proximoPC + deslocamento; a 161 usa B + deslocamento; o modo indexado soma X.

4. decodificador.c | linhas 182-207
Indireto versus imediato. No indireto, a memória fornece o endereço final. No imediato, o campo vira o próprio valor do operando.

FALA SUGERIDA

Minha parte transforma os bytes da memória em uma instrução que o simulador consegue executar. Em decodificador.c, primeiro identificamos o formato e separamos os campos. No formato 2, os dois grupos de quatro bits representam os registradores. Nos formatos 3 e 4, extraímos o opcode e os bits de endereçamento.

O formato 3 usa três bytes e um deslocamento de 12 bits. O formato 4 usa quatro bytes e permite um endereço de 20 bits. Também calculamos o próximo PC de acordo com esse tamanho.

Depois vem o cálculo do endereço efetivo. No modo relativo ao PC, somamos o deslocamento com sinal ao próximo PC, e não ao endereço da instrução atual. No relativo à base, usamos B; no indexado, acrescentamos X.

No modo imediato, o operando já é o valor. No indireto, buscamos na memória o endereço que será usado. O decodificador também rejeita combinações inválidas de flags e leituras fora da memória. Ele interpreta a instrução; a operação em si fica com os módulos de execução.

LEMBRETE PARA A GRAVAÇÃO
Ao mostrar o PC relativo, destaque proximoPC na linha 154. É o detalhe mais importante para explicar por que os saltos relativos chegam ao lugar correto.

PARTE 03 | TEMPO SUGERIDO: 1MIN20S

Gabriel Britto

Instruções lógicas, aritméticas e ponto flutuante

ABRIR NA TELA
Aritimetica.c
sim_real.c (apoio para o código de condição)

TRECHOS PARA MOSTRAR, NESTA ORDEM

1. Aritimetica.c | linhas 13-29
Limite de 24 bits e comparação. limitar_24_bits preserva o sinal; atualizar_codigo_condicional registra menor, igual ou maior.

2. Aritimetica.c | linhas 43-72
Operações principais. Mostre ADD, SUB, MUL, DIV, AND, OR e COMP. COMP atualiza a condição sem substituir o acumulador.

3. Aritimetica.c | linhas 131-149
Deslocamentos de bits. SHIFTL faz rotação em 24 bits; SHIFTR preserva o sinal. Não descreva os dois como uma simples multiplicação ou divisão.

4. Aritimetica.c | linhas 154-164
Conversões e limite atual. FIX e FLOAT convertem entre os acumuladores; NORM ainda está vazio. As operações com double aparecem nas linhas 76-96.

FALA SUGERIDA

Minha parte reúne as operações aritméticas e lógicas. Em Aritimetica.c, limitar_24_bits mantém os resultados no tamanho de uma palavra e interpreta o bit de sinal. Isso é necessário porque os tipos da linguagem C não têm exatamente o mesmo tamanho dos registradores simulados.

As funções ADD, SUB, MUL e DIV calculam os resultados. AND e OR fazem operações bit a bit. Já COMP compara os operandos e altera o código de condição, sem trocar o valor do acumulador. Essa condição será usada nos desvios condicionais. Na integração, sim_real.c traduz a condição local para o registrador SW.

Também existem deslocamentos: SHIFTL faz uma rotação dentro dos 24 bits, enquanto SHIFTR mantém o sinal ao deslocar para a direita.

Sobre ponto flutuante, existem rotinas usando double e conversões FIX e FLOAT, mas essa parte é preliminar: NORM ainda está vazio e a representação SIC/XE de 48 bits não está validada. Por isso, a demonstração será feita com operações inteiras. O João Victor mostra agora a movimentação dos dados e os desvios.

LEMBRETE PARA A GRAVAÇÃO
Importante: não apresentar ponto flutuante como completo. Em sim_real.c, linhas 75-77, confira a tradução da condição para SW; as conversões de 48 bits ficam nas linhas 95-106.

PARTE 04 | TEMPO SUGERIDO: 1MIN25S

João Victor

Movimentação de dados e desvios

ABRIR NA TELA
instrucoes_movimentacao_desvios.c
sim_real.c (linha 653, apoio à chamada de subrotina)

TRECHOS PARA MOSTRAR, NESTA ORDEM

1. instrucoes_movimentacao_desvios.c | linhas 34-70
Carregar uma palavra. ler_operando_palavra distingue imediato e memória. carregar_palavra grava o resultado no registrador de destino.

2. instrucoes_movimentacao_desvios.c | linhas 72-102
LDCH e armazenamento. LDCH troca só o byte inferior de A; armazenar_palavra grava três bytes e rejeita destino imediato.

3. instrucoes_movimentacao_desvios.c | linhas 120-156
Saltos e subrotinas. J escreve PC; saltar_se consulta a condição; JSUB salva o retorno em L e RSUB recupera esse valor para PC.

4. instrucoes_movimentacao_desvios.c | linhas 230-265
Despacho das instruções. Mostre CLEAR, RMO e os casos dos desvios no switch. CLEAR zera; RMO copia de r1 para r2.

FALA SUGERIDA

Minha parte implementa as instruções de movimentação de dados e controle de fluxo. Em instrucoes_movimentacao_desvios.c, as cargas usam uma rotina comum: ela verifica se o operando é imediato ou vem da memória e escreve no registrador de destino. Assim, instruções como LDA e LDX reaproveitam a mesma lógica.

Nas operações de armazenamento, como STA, fazemos o caminho contrário: o conteúdo do registrador vai para a memória. Não aceitamos um destino imediato, porque ele representa um valor, não uma posição para escrever. LDCH é um caso específico: modifica apenas o byte inferior de A e preserva os demais.

No controle de fluxo, J altera o PC diretamente. JEQ, JGT e JLT verificam o código de condição antes de saltar. JSUB guarda em L o endereço de retorno e vai para a subrotina. RSUB copia L de volta para PC. Isso funciona porque o executor já avançou o PC antes de chamar este módulo.

Por fim, o switch encaminha cada opcode para sua operação. CLEAR zera um registrador e RMO copia entre registradores. Esse módulo acessa a memória e os registradores por funções de integração, sem criar outro estado separado.

LEMBRETE PARA A GRAVAÇÃO
Destaque as linhas 145-147: o retorno de JSUB é o PC já avançado. Em sim_real.c, linha 653, mostre onde esse avanço acontece antes da execução.

PARTE 05 | TEMPO SUGERIDO: 1MIN25S

Yago

Carregamento pela interface e montador

ABRIR NA TELA
programa.c
montagem/passos.c
montagem/objeto.c

TRECHOS PARA MOSTRAR, NESTA ORDEM

1. programa.c | linhas 13-31
Da interface ao programa carregado. Arquivo .asm passa pelo montador e vira imagem de memória; outros arquivos seguem para o carregador existente.

2. montagem/passos.c | linhas 59-102
Primeiro passo do montador. Mostre símbolos, contador de localização e reservas. Os rótulos recebem endereços antes da geração final dos bytes.

3. montagem/passos.c | linhas 199-245
Segundo passo. Resolve operandos e gera a imagem: BYTE, WORD e instruções. BASE e END também são tratados aqui.

4. montagem/objeto.c | linhas 5-18
Saída de arquivo objeto. Mostre os registros H, T e E. O montador de linha de comando pode salvar o objeto; a interface recebe a imagem diretamente.

FALA SUGERIDA

Vou mostrar como o código-fonte chega à execução pela interface. Em programa.c, o carregamento identifica arquivos assembly pela extensão. Quando é um arquivo .asm, ele chama o montador e, se não houver erro, entrega a imagem gerada ao simulador. Assim, a interface não precisa executar um montador externo.

O montador trabalha em dois passos. No primeiro, em montagem/passos.c, percorre as linhas, controla os endereços e registra os símbolos. Isso permite usar um rótulo antes de ele aparecer no arquivo, porque seu endereço será conhecido antes da geração final.

No segundo passo, resolve os operandos e gera os bytes das instruções e dos dados. Diretivas como WORD e RESW têm funções diferentes: uma produz um valor e a outra reserva espaço.

Em montagem/objeto.c, temos a gravação dos registros de cabeçalho, texto e entrada, usados na saída do montador de linha de comando. Pela interface, a carga é feita diretamente na memória. O escopo atual é montagem absoluta de um subconjunto de instruções inteiras; não estamos apresentando um ligador ou processador de macros. Agora o Vitor mostra a visualização dessa memória.

LEMBRETE PARA A GRAVAÇÃO
Divisão sugerida para o vídeo: o grupo informou Yago na interface, mas não detalhou sua função. Confirmem esta fala de integração/montador antes de gravar; ela não atribui autoria do código.

PARTE 06 | TEMPO SUGERIDO: 1MIN10S

Vitor

Apresentação gráfica da memória

ABRIR NA TELA
memview.c
main.c (linhas 111-114, captura antes do passo)

TRECHOS PARA MOSTRAR, NESTA ORDEM

1. memview.c | linhas 25-36
Fotografia antes da instrução. A cópia com memcpy guarda a memória antes do passo. Ela será comparada ao estado atual para destacar mudanças.

2. memview.c | linhas 49-75
Organização da grade. Mostre o cálculo das linhas visíveis, os cabeçalhos e o endereço inicial de cada linha da memória.

3. memview.c | linhas 82-95
Cores e valores. Linha 88 compara os bytes; linha 90 destaca alterações; linha 91 contorna o endereço do PC; linha 94 formata em hexadecimal.

4. memview.c | linhas 38-47
Limpeza e liberação. Esquecer a cópia remove os destaques; liberar a memória auxiliar evita manter a alocação depois do uso.

FALA SUGERIDA

Minha parte apresenta a memória visualmente. Em memview.c, organizamos os bytes em uma grade de dezesseis colunas. Cada linha mostra seu endereço inicial e cada célula apresenta o valor de um byte em hexadecimal. Isso permite relacionar os endereços do programa com o conteúdo real da memória.

Antes de executar uma instrução, guardamos uma cópia do estado anterior. Na hora de desenhar, comparamos cada byte atual com essa cópia. Se o valor mudou, a célula recebe um destaque amarelo. O endereço indicado pelo PC também recebe um contorno próprio.

A quantidade de linhas visíveis depende do espaço disponível no painel. A renderização ainda verifica os limites para não tentar mostrar uma posição fora da memória.

Essa parte apenas lê e desenha o estado: ela não executa as instruções. Na demonstração, depois de STA, conseguimos ver o resultado da soma escrito na memória. O André vai mostrar como essa visualização se conecta aos controles da aplicação.

LEMBRETE PARA A GRAVAÇÃO
O amarelo indica diferença em relação ao estado capturado antes do passo, não um histórico de todas as alterações. A captura é chamada em main.c, linhas 111-114.

PARTE 07 | TEMPO SUGERIDO: 1MIN40S

André

Layout principal e integração do sistema

ABRIR NA TELA
main.c
sim_real.c
exemplos/soma.asm (deixar aberto para a demonstração)

TRECHOS PARA MOSTRAR, NESTA ORDEM

1. main.c | linhas 108-128
Executar um passo. Guarda o estado visual, chama sim_passo e contabiliza a execução. Em caso de término ou erro, interrompe a execução automática.

2. sim_real.c | linhas 642-669
Ciclo de execução. Busca PC, decodifica, avança PC, executa e sincroniza o estado apresentado. É a ligação entre os módulos do processador.

3. main.c | linhas 163-177
Reiniciar com segurança. Restaura a imagem inicial do programa e seu ponto de entrada. O reset não deve deixar o resultado anterior na memória.

4. main.c | linhas 368-378
Laço da interface. Processa entradas e redesenha barra, registradores, memória e log. Os painéis são funções separadas no mesmo ciclo gráfico.

FALA SUGERIDA

Minha parte organiza a tela e integra os módulos. Em main.c, o laço principal recebe as ações do usuário e desenha a barra de controles, os registradores, a memória e o log. A interface usa Raylib, mas a lógica de execução fica separada do desenho.

Quando clicamos em Passo, dar_passo guarda o estado anterior e chama sim_passo. Em sim_real.c, essa função decodifica a instrução, avança o PC, executa a operação e atualiza o estado exibido. A execução automática repete esse processo; o reinício restaura a imagem original do programa.

Agora vou demonstrar com soma.asm. Primeiro, LDA carrega cinco em A; depois ADD soma três. No terceiro passo, STA escreve oito na posição de resultado, que aparece destacada na memória.

Ao continuar, a comparação faz o desvio correto, a subrotina coloca sete em B e RSUB retorna usando L. No final, temos A igual a oito, B igual a sete e nove instruções executadas. Neste simulador, o salto para o próprio endereço é a convenção de término. Por fim, Reiniciar restaura o estado inicial, mostrando que a interface, o montador e a execução estão conectados.

LEMBRETE PARA A GRAVAÇÃO
Reserve cerca de 30 segundos deste bloco para a demonstração. Use a sequência da página 9. O salto para si mesmo é uma convenção do simulador, não uma instrução HALT do SIC/XE.

Demonstração conjunta

Exemplo: exemplos/soma.asm | resultado esperado e checklist

PREPARAR ANTES DE GRAVAR
Abra simulador-gui.exe a partir da pasta completa extraída. Confira que exemplos/soma.asm está carregado e que a execução está reiniciada. Deixe o arquivo aberto no editor ao lado da aplicação.

SEQUÊNCIA DE TELA

1. Mostrar o fonte
soma.asm, linhas 3-5: LDA #5, ADD #3 e STA RESULT. A reserva RESULT aparece na linha 13.

2. Clicar em Passo uma vez
A fica 000005 e PC fica 000003. Relacione o resultado com a carga imediata.

3. Clicar em Passo mais uma vez
A fica 000008 e PC fica 000006. É o resultado de 5 + 3.

4. Clicar em Passo pela terceira vez
STA grava 00 00 08 nos endereços 00001E-000020; PC fica 000009. O byte 000020 muda para 08 e aparece destacado.

5. Continuar até o término
COMP e JEQ pulam LDA #99. JSUB chama ROTINA, LDB carrega 7 e RSUB retorna. O salto J FIM encerra pela convenção do simulador.

6. Mostrar Reiniciar
O simulador restaura a imagem inicial, os registradores e o ponto de entrada. A palavra RESULT volta a zero.

CONFERÊNCIA AO FINAL: 9 instruções executadas
A = 000008 B = 000007 L = 000015
PC = 000015 SW = 000002 RESULT = 00 00 08
Valores exibidos em hexadecimal. SW = 2 representa a condição de igualdade nesta integração.

CHECKLIST RÁPIDO

- Aumentar a fonte do editor e manter os números de linha visíveis.
- Não gravar com outra cópia do projeto: as referências são desta versão.
- Confirmar a divisão sugerida do Yago e ensaiar as transições.
- Não afirmar suporte completo a ponto flutuante, macros ou ligação externa.
- Mostrar o resultado na interface, não apenas o código no editor.

Referência técnica: fontes locais do pacote simulador-sicxe-interface. Os trechos numerados foram conferidos na preparação deste roteiro; não houve alteração nos arquivos do projeto para produzir o PDF.